

Template Implementation Quality Comparison

Date: 2026-05-23
Project: /home/ubuntu/dnd35prototype
Scope: Comparing Celestial/Fiendish template architecture vs Skeleton/Zombie/Lycanthrope template architecture

1. Executive Summary

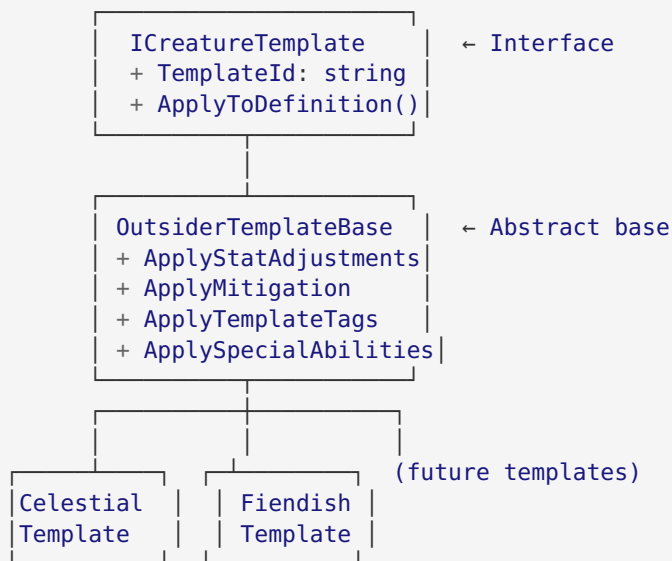
The project has **two distinct template architectures** that serve different use cases well. The Celestial/Fiendish system is more architecturally sophisticated (interface-based, registry-driven, runtime application), while the Skeleton/Zombie/Lycanthrope system is more pragmatically appropriate for its use case (pre-built stat blocks for standalone encounter creatures). Neither is “wrong” — they solve different problems. However, there are concrete improvements that could unify the systems and reduce code duplication.

Quality Scores (1-10)

Metric	Celestial/Fiendish	Skeleton/Zombie/Lycanthrope
Modularity	9	6
Extensibility	9	7
Code Reuse	8	4
Composability	8	2
Testability	8	7
D&D Rules Accuracy	9	9
Pragmatic Fitness	7	9
Overall	8.3	6.3

2. Architecture Overview

2.1 Celestial/Fiendish: Interface-Based Registry Pattern



```

CreatureTemplateRegistry (static dictionary)
├── "celestial" → CelestialTemplate
├── "fiendish" → FiendishTemplate
+ ResolveTemplates(NPCDefinition)
+ ApplyTemplatesClone(NPCDefinition)
    
```

Application flow:

```

NPCDefinition.AppliedTemplateIds = { "fiendish" }
    
```

↓ (at spawn time)

```

CreatureTemplateRegistry.ApplyTemplatesClone(def)
    
```

↓

```

Clone → Resolve templates → Apply each → Return mutated clone
    
```

Key files:

- Assets/Scripts/Character/Templates/CreatureTemplateFramework.cs (258 lines)
- Assets/Scripts/CombatSystems/TemplateSmiteSystem.cs (247 lines)

2.2 Skeleton/Zombie/Lycanthrope: Static Method + Factory Pattern

```

SkeletonTemplate (static class)
├─ Apply(baseCreature, ...)
├─ GetSkeletonNaturalArmor()
├─ GetSkeletonClawDamageDice()
├─ GetSkeletonCR()
├─ StripSpecialEffects() ← DUPLICATED →
ZombieTemplate (static class)
├─ Apply(baseCreature, ...)
├─ GetZombieNaturalArmorIncrease()
├─ GetZombieSlamDamageDice()
├─ GetZombieCR()
├─ StripSpecialEffects()

SkeletonFactory (static class)
├─ CreateFromRegistered()
├─ HumanWarriorSkeleton()
├─ WolfSkeleton()
├─ OwlbearSkeleton()
├─ MinotaurSkeleton()
├─ MegaraptorSkeleton()
├─ HorseSkeleton()
├─ TrollSkeleton()
ZombieFactory (static class)
├─ CreateFromRegistered()
├─ HumanCommonerZombie()
├─ HumanWarriorZombie()
├─ TroglodyteZombie()
├─ OgreZombie()
├─ MinotaurZombie()
├─ OwlbearZombie()
├─ BugbearZombie()

LycanthropeTemplate (static class)
├─ Apply(baseCreature, animalType, isNatural, ...)
├─ GetAnimalModifiers(animalType)
├─ GetHybridClawDamageDice()
├─ CalculateLycanthropeCR()
├─ (no StripSpecialEffects – clears attacks differently)

LycanthropeFactory (static class)
├─ CreateFromRegistered()
├─ Werewolf()
├─ WerewolfLord()
├─ Wererat()
├─ Wereboar()
├─ Weretiger()
├─ Werebear()
├─ DireWereboar()
├─ AfflictedWerewolf()

Application flow (BUILD-TIME):
Factory.Werewolf() → creates base def inline → LycanthropeTemplate.Apply() →
NPCDefinition
  ↓ (at init)
NPCDatabase.Register(result)
  ↓ (at spawn time)
NPCDatabase.Get("werewolf") → already-baked stat block

```

Key files:

- Assets/Scripts/Character/SkeletonTemplate.cs (786 lines)
 - Assets/Scripts/Character/ZombieTemplate.cs (757 lines)
 - Assets/Scripts/Character/LycanthropeTemplate.cs (1124 lines)
 - Assets/Scripts/Character/NPCDatabase_Skeletons.cs
 - Assets/Scripts/Character/NPCDatabase_Zombies.cs
 - Assets/Scripts/Character/NPCDatabase_Lycanthropes.cs
-

3. Detailed Comparison

3.1 Interface-Based vs Static Methods

Aspect	Celestial/Fiendish	Skeleton/Zombie/Lycanthrope
Pattern	<code>ICreatureTemplate</code> interface with <code>ApplyToDefinition()</code>	Static <code>Apply()</code> methods on static classes
Polymorphism	✓ Templates are interchangeable via interface	✗ Each template is a separate, unrelated static class
OCP (Open/Closed)	✓ Add new templates by implementing <code>ICreatureTemplate</code>	⚠ Must create entirely new static classes
Discoverability	✓ Registry lists all templates	✗ Must know class names

Impact: The interface pattern allows the system to treat all templates uniformly. The static pattern means each template is its own island — no way to iterate over “all templates” or compose them generically.

3.2 Registry System vs Direct Calls

Aspect	Celestial/Fiendish	Skeleton/Zombie/Lycanthrope
Registration	<code>CreatureTemplateRegistry</code> static dictionary	None — Factory methods called directly
Lookup	By string ID: "celestial", "fiendish"	By factory method: <code>SkeletonFactory.HumanWarriorSkeleton()</code>
Dynamic application	✓ Any creature can be templated by adding ID to <code>AppliedTemplateIds</code>	✗ Must manually call Factory or <code>Template.Apply()</code>
Multi-template	✓ <code>AppliedTemplateIds</code> is a List — compose multiple	✗ Not supported without manual chaining

Impact: The registry pattern is critical for the Summon Monster system — it needs to tag any arbitrary creature with a template at runtime. The static pattern works fine for pre-built encounter creatures but can’t dynamically template a creature without code changes.

3.3 Runtime Application vs Build-Time Baking

Aspect	Celestial/Fiendish	Skeleton/Zombie/Lycanthrope
When applied	At spawn time via <code>ApplyTemplatesClone()</code>	At init time, baked into <code>NPCDatabase.Register()</code>
Base creature needed?	At spawn time — reads from DB	Only at factory creation time
Re-application	✓ Fresh clone each spawn	N/A — already baked
Hot-swap	✓ Change template logic, all spawns update	✗ Must re-run factory

Impact: Runtime application is more flexible but slightly more expensive per spawn. Build-time baking is $O(1)$ at spawn but inflexible. For the Skeleton/Zombie/Lycanthrope use case, build-time is actually **more appropriate** — you don't typically need to apply a skeleton template to an arbitrary creature at runtime (though Animate Dead spell might want this eventually).

3.4 Composability — Can You Apply Multiple Templates?

Scenario	Celestial/Fiendish	Skeleton/Zombie/Lycanthrope
Fiendish + Celestial	✓ Theoretically (via <code>AppliedTemplateIds</code> list)	N/A
Fiendish Skeleton	✗ Not integrated — Skeleton doesn't use registry	✗ Would need manual chaining
Skeleton Werewolf	✗ Not integrated	✗ Would need manual chaining
Fiendish Wolf	✓ Works perfectly via tag	N/A — but could via <code>SkeletonTemplate.Apply()</code> on a fiendish wolf

Impact: Template composition (e.g., “Fiendish Skeleton”) is a genuine D&D concept but a niche use case. The current split architectures make this impossible without manual intervention. Unifying under the `ICreatureTemplate` interface would enable it.

3.5 Code Reuse

Duplication	Details
<code>StripSpecialEffects()</code>	Identical 15-line method in both <code>SkeletonTemplate.cs</code> and <code>ZombieTemplate.cs</code>
Base creature inline definitions	<code>base_owlbear</code> defined identically in <code>SkeletonFactory.OwlbearSkeleton()</code> AND <code>ZombieFactory.OwlbearZombie()</code>
<code>base_minotaur</code> definition	Identical in <code>SkeletonFactory.MinotaurSkeleton()</code> AND <code>ZombieFactory.MinotaurZombie()</code>
<code>base_human_warrior</code> definition	Near-identical in <code>SkeletonFactory.HumanWarriorSkeleton()</code> , <code>ZombieFactory.HumanWarriorZombie()</code> , and <code>LycanthropeFactory.Werewolf()</code>
Undead boilerplate	~25 lines of undead setup (immunities, mindless, no spells, etc.) duplicated between <code>Skeleton</code> and <code>Zombie Apply()</code>
Natural attack size tables	3 separate sets (<code>Skeleton</code> claw, <code>Zombie</code> slam, <code>Lycanthrope</code> claw/bite) that could share a utility

Impact: The `OutsiderTemplateBase` in `Celestial/Fiendish` shares ALL common logic via the abstract base class. The static templates have zero shared code — even identical helper methods are copy-pasted.

3.6 Extensibility — How Easy to Add New Templates?

Adding a new...	Celestial/Fiendish	Skeleton/Zombie/Lycanthrope
Template type (e.g., “Half-Dragon”)	✅ Implement <code>ICreatureTemplate</code> , add to registry (2 places)	⚠️ Create new static class + factory from scratch
Template variant (e.g., new skeleton type)	N/A	✅ Add new factory method (easy within existing pattern)
Template feature (e.g., energy resistance)	✅ Add to <code>OutsiderTemplateBase</code> — all templates inherit	❌ Must add to each static class independently

3.7 Testability

Aspect	Celestial/Fiendish	Skeleton/Zombie/Lycanthrope
Unit test	✓ Can test via interface mock	✓ Can test static methods directly
Integration test	✓ Test encounters exist	✓ Encounter presets cover all variants
Stat verification	⚠ Stats computed at runtime (harder to verify at build time)	✓ Stats baked at init — can verify via Python scripts
Regression	✓ Registry-based, easy to add new templates to tests	✓ Factory-based, easy to add new variants

Both are reasonably testable. The static pattern is actually **easier to test** in some ways because the stats are deterministic at build time.

4. What Makes Celestial/Fiendish Higher Quality?

4.1 Architectural Strengths

1. **Single Responsibility:** `ICreatureTemplate` has ONE job — mutate a definition. No factory, no registration mixing.
2. **Shared base class:** `OutsiderTemplateBase` eliminates duplication between Celestial and Fiendish.
3. **Registry pattern:** `CreatureTemplateRegistry` provides a central lookup, enabling dynamic template application.
4. **Composition-ready:** `AppliedTemplateIds` is a list, not a single value — multiple templates can stack.
5. **Spawn-time application:** The `ApplyTemplatesClone()` call in `GameManager.cs` and `GameManager.SpellCasting.cs` means ANY creature can be templated without pre-registration.
6. **Helper deduplication:** `AddOrRaiseResistance()`, `AddTag()`, `AddSpecialAbility()`, `AddTemplateId()` are all shared utility methods.
7. **Full combat system:** `TemplateSmiteSystem.cs` (247 lines) provides complete Smite Evil/Good with UI, AI, and alignment validation — deeply integrated.

4.2 Design Patterns Used

- **Strategy Pattern** (via `ICreatureTemplate`)
- **Registry Pattern** (via `CreatureTemplateRegistry`)
- **Template Method Pattern** (via `OutsiderTemplateBase` abstract class with protected hooks)
- **Prototype Pattern** (via `NPCDefinition.Clone()` in `ApplyTemplatesClone()`)

5. What's Missing from Skeleton/Zombie/Lycanthrope?

5.1 Concrete Gaps

Gap	Severity	Details
No <code>ICreatureTemplate</code> integration	Medium	Templates exist outside the registry — cannot be applied dynamically
Duplicated <code>StripSpecialEffects()</code>	Low	15 identical lines in 2 files
Duplicated base creature definitions	Medium	<code>base_owlbear</code> , <code>base_minotaur</code> , <code>base_human_warrior</code> defined inline 2-3 times each
No shared undead template base	Medium	~25 lines of undead boilerplate (immunities, mindless, etc.) duplicated
No shared natural attack size utility	Low	3 separate size→damage tables that follow the same pattern
No dynamic application	Medium	Can't apply skeleton/zombie template at runtime (e.g., <code>Animate Dead</code> spell)
No composition support	Low-Medium	Can't create "Fiendish Skeleton" without manual code
No registry participation	Medium	<code>CreatureTemplateRegistry</code> doesn't know about skeleton/zombie/lycanthrope templates

5.2 What's Actually Fine

Aspect	Assessment
D&D 3.5e rules accuracy	✓ Excellent — all three templates follow MM rules precisely
Factory pattern quality	✓ Clean, well-organized, good inline documentation
Encounter presets	✓ Good variety and encounter level balance
Error handling	✓ Null checks, fallback definitions, Debug.LogWarning
Code documentation	✓ Comprehensive XML docs with MM page references
Visual integration	✓ Custom colors for each template type
Template tracking	✓ <code>AppliedTemplateIds</code> is set (even though not used by registry)

6. Detailed Refactoring Plan

Phase 1: Extract Shared Utilities (Effort: ~1 hour)

Goal: Eliminate code duplication without changing architecture.

Step 1.1: Create UndeadTemplateUtils.cs

```
// Assets/Scripts/Character/Templates/UndeadTemplateUtils.cs
public static class UndeadTemplateUtils
{
    /// <summary>
    /// Strip all on-hit special effects from a natural attack.
    /// Used by both Skeleton and Zombie templates.
    /// </summary>
    public static void StripSpecialEffects(NaturalAttackDefinition attack) { ... }

    /// <summary>
    /// Apply standard undead modifications shared by all undead templates.
    /// Type → Undead, no CON/INT, WIS 10, CHA 1, undead immunities,
    /// IsMindless, no spells, etc.
    /// </summary>
    public static void ApplyBaseUndeadTraits(NPCDefinition def) { ... }

    /// <summary>
    /// Clear all special attacks from a creature definition.
    /// </summary>
    public static void ClearAllSpecialAttacks(NPCDefinition def) { ... }

    /// <summary>
    /// Standard natural attack damage by size category.
    /// </summary>
    public static (int dice, int count) GetNaturalAttackDamage(
        SizeCategory size, NaturalAttackType type) { ... }
}
```

Step 1.2: Update SkeletonTemplate.cs and ZombieTemplate.cs

Replace duplicated methods with calls to `UndeadTemplateUtils`.

```
// Before (in both files):
private static void StripSpecialEffects(NaturalAttackDefinition attack) { ... }

// After:
// Delete private method, use: UndeadTemplateUtils.StripSpecialEffects(attack)
```

Step 1.3: Create `BaseCreatureDefinitions.cs`

```
// Assets/Scripts/Character/Templates/BaseCreatureDefinitions.cs
public static class BaseCreatureDefinitions
{
    /// <summary>
    /// Human Warrior 1 – used by Skeleton, Zombie, and Lycanthrope templates.
    /// </summary>
    public static NPCDefinition HumanWarrior1() { ... }

    /// <summary>
    /// Owlbear – used by both Skeleton and Zombie factories.
    /// </summary>
    public static NPCDefinition Owlbear() { ... }

    /// <summary>
    /// Minotaur – used by both Skeleton and Zombie factories.
    /// </summary>
    public static NPCDefinition Minotaur() { ... }
}
```

Phase 2: Integrate with `ICreatureTemplate` (Effort: ~2-3 hours)

Goal: Make Skeleton/Zombie/Lycanthrope templates participate in the `CreatureTemplateRegistry`, enabling dynamic application.

Step 2.1: Create Wrapper Classes

```
// Assets/Scripts/Character/Templates/SkeletonCreatureTemplate.cs
public sealed class SkeletonCreatureTemplate : ICreatureTemplate
{
    public string TemplateId => "skeleton";

    public void ApplyToDefinition(NPCDefinition definition)
    {
        // Delegate to existing static method
        // but apply IN-PLACE since ApplyTemplatesClone already cloned
        NPCDefinition result = SkeletonTemplate.Apply(definition);
        if (result == null) return;

        // Copy all fields from result back to definition
        // (or refactor Apply to mutate in-place with an overload)
        CopyFields(result, definition);
    }
}

// Similar for ZombieCreatureTemplate and LycanthropeCreatureTemplate
```

Alternative (cleaner): Refactor `SkeletonTemplate.Apply()` to have an in-place overload:

```

public static class SkeletonTemplate
{
    // Existing: returns new clone
    public static NPCDefinition Apply(NPCDefinition baseCreature, ...) { ... }

    // New: mutates in place (for registry integration)
    public static void ApplyInPlace(NPCDefinition definition, bool hasHands = true, bool hasWings = false)
    {
        // Same logic but operates on the passed definition directly
        // (the clone is already done by ApplyTemplatesClone)
    }
}

```

Step 2.2: Register in CreatureTemplateRegistry

```

// In CreatureTemplateFramework.cs, add to the dictionary:
private static readonly Dictionary<string, ICreatureTemplate> _templates = new Dictionary<...>
{
    { "celestial", new CelestialTemplate() },
    { "fiendish", new FiendishTemplate() },
    { "skeleton", new SkeletonCreatureTemplate() }, // NEW
    { "zombie", new ZombieCreatureTemplate() },    // NEW
    { "lycanthrope", new LycanthropeCreatureTemplate() }, // NEW
};

```

Step 2.3: Enable Runtime Template Application

This enables future systems like **Animate Dead** spell:

```

// In spell resolution:
NPCDefinition target = NPCDatabase.Get(corpseCreatureId);
target.AppliedTemplateIds = new List<string> { "zombie" };
NPCDefinition zombie = CreatureTemplateRegistry.ApplyTemplatesClone(target);
// Spawn zombie at corpse location

```

Phase 3: Enable Template Composition (Effort: ~1-2 hours)

Goal: Support stacking templates (e.g., "Fiendish Skeleton").

Step 3.1: Define Application Order

```

// Templates should be applied in a specific order:
// 1. Creature-type templates first (Skeleton, Zombie) – these radically change the creature
// 2. Enhancement templates second (Celestial, Fiendish) – these add on top
// 3. Subtype templates last (Lycanthrope) – these merge creature + animal

// Add an Order property to ICreatureTemplate:
public interface ICreatureTemplate
{
    string TemplateId { get; }
    int ApplicationOrder { get; } // Lower = applied first
    void ApplyToDefinition(NPCDefinition definition);
}

```

Step 3.2: Update Registry to Sort

```
public static NPCDefinition ApplyTemplatesClone(NPCDefinition source)
{
    NPCDefinition clone = source.Clone();
    List<ICreatureTemplate> templates = ResolveTemplates(clone);
    templates.Sort((a, b) => a.ApplicationOrder.CompareTo(b.ApplicationOrder));
    for (int i = 0; i < templates.Count; i++)
        templates[i].ApplyToDefinition(clone);
    return clone;
}
```

Step 3.3: Handle Template Conflicts

```
// Fiendish/Celestial need to handle undead CON gracefully (already done):
int conBonus = string.Equals(definition.CreatureType, "Undead", ...) ? 0 : 2;

// Skeleton/Zombie need to handle pre-existing template resistances:
// Instead of replacing DamageResistances, use AddOrRaiseResistance pattern
```

Phase 4: Maintain Backward Compatibility (Effort: ~30 min)

Goal: Ensure existing factory registrations still work.

1. **Keep all Factory classes** — they continue to work as before for pre-built encounters
2. **Keep all RegisterCreatures_X() methods** — they still register pre-built stat blocks
3. **Add AppliedTemplateIds** tracking to all factory-created creatures (already done!)
4. **Registry integration is additive** — it enables new capabilities without breaking existing ones

7. Estimated Total Effort

Phase	Effort	Priority	Impact
Phase 1: Extract shared utilities	~1 hour	High	Eliminates duplication, improves maintainability
Phase 2: ICreatureTemplate integration	~2-3 hours	Medium	Enables dynamic template application (Animate Dead)
Phase 3: Template composition	~1-2 hours	Low	Niche but cool (Fiendish Skeleton combos)
Phase 4: Backward compatibility	~30 min	High	Must-do alongside any refactoring
Total	~5-6 hours		

8. Recommendation

Do Now (Quick Wins):

1. **✓ Extract `StripSpecialEffects()` to shared utility** — eliminates the most obvious duplication
2. **✓ Extract base creature definitions** — stops defining `base_owlbear` in 2 places

Do When Needed:

1. **ICreatureTemplate wrappers** — Do this when implementing **Animate Dead** spell (which needs runtime skeleton/zombie application)
2. **Template composition** — Do this if/when the game needs “Fiendish Skeleton” type combos

Don't Do (Not Worth It):

1. **Full rewrite of Skeleton/Zombie/Lycanthrope** to use abstract base classes — The static pattern is clear, well-documented, and works well for pre-built encounters. The Celestial/Fiendish pattern is better architecture, but the static pattern isn't broken.
-

9. Why the Difference Exists (And Why It's OK)

The two architectures solve **fundamentally different problems**:

Celestial/Fiendish needs to template **any arbitrary creature** at runtime. A Celestial Wolf, Celestial Eagle, Fiendish Rat, Fiendish Giant — the base creature is selected dynamically by the Summon Monster spell. The template must be generic enough to work on anything.

Skeleton/Zombie/Lycanthrope provides **specific encounter creatures**. You fight “a Human Warrior or Skeleton” or “a Werewolf” — these are pre-defined stat blocks with specific equipment, descriptions, and encounter balance. The template is a build-time tool for the designer, not a runtime system.

The Lycanthrope template is a middle ground — it's more complex than Skeleton/Zombie (merging two creatures) but still used for pre-built encounters. It could benefit from registry integration if a “curse of lycanthropy” system ever needs to apply the template to PCs or arbitrary NPCs at runtime.

The Bridge: `AppliedTemplateIds`

All three static templates already set `AppliedTemplateIds` on their output:

- Skeletons set `{ "skeleton" }`
- Zombies set `{ "zombie" }`
- Lycanthropes set `{ "lycanthrope", "[prefix]" }`

This means they're **already tracking which templates were applied**, even though the registry doesn't know about them yet. This makes Phase 2 integration straightforward — just add wrapper classes and register them.

10. Code Examples: Before and After

Before: Duplicated StripSpecialEffects

```
// SkeletonTemplate.cs line 372-387
private static void StripSpecialEffects(NaturalAttackDefinition attack)
{
    if (attack == null) return;
    attack.PoisonOnHitId = null;
    attack.ParalysisOnHitDC = 0;
    // ... 11 more lines ...
}

// ZombieTemplate.cs line 354-370 – IDENTICAL copy
private static void StripSpecialEffects(NaturalAttackDefinition attack)
{
    if (attack == null) return;
    attack.PoisonOnHitId = null;
    attack.ParalysisOnHitDC = 0;
    // ... 11 more lines ...
}
```

After: Shared Utility

```
// UndeadTemplateUtils.cs
public static class UndeadTemplateUtils
{
    public static void StripSpecialEffects(NaturalAttackDefinition attack) { ... }
}

// SkeletonTemplate.cs
UndeadTemplateUtils.StripSpecialEffects(skel.NaturalAttacks[i]);

// ZombieTemplate.cs
UndeadTemplateUtils.StripSpecialEffects(zombie.NaturalAttacks[i]);
```

Before: Duplicated Base Creature

```
// SkeletonFactory.OwlbearSkeleton() – line 537
var baseDef = new NPCDefinition {
    Id = "base_owlbear", Name = "Owlbear", HitDice = 5,
    SizeCategory = SizeCategory.Large, STR = 21, DEX = 12, CON = 21, ...
};

// ZombieFactory.OwlbearZombie() – line 669
var baseDef = new NPCDefinition {
    Id = "base_owlbear", Name = "Owlbear", HitDice = 5,
    SizeCategory = SizeCategory.Large, STR = 21, DEX = 12, CON = 21, ...
};
```

After: Shared Definition

```
// BaseCreatureDefinitions.cs
public static NPCDefinition Owlbear() => new NPCDefinition {
    Id = "base_owlbear", Name = "Owlbear", HitDice = 5,
    SizeCategory = SizeCategory.Large, STR = 21, DEX = 12, CON = 21, ...
};

// SkeletonFactory.cs
var skel = SkeletonTemplate.Apply(BaseCreatureDefinitions.Owlbear(), ...);

// ZombieFactory.cs
var zombie = ZombieTemplate.Apply(BaseCreatureDefinitions.Owlbear(), ...);
```

11. Conclusion

The Celestial/Fiendish template system is architecturally superior in terms of modularity, extensibility, and composability. However, the Skeleton/Zombie/Lycanthrope system is pragmatically well-suited for its use case of pre-built encounter creatures. The main improvements needed are:

1. **Immediate:** Extract shared utilities to eliminate code duplication (~1 hour)
2. **When needed:** Add `ICreatureTemplate` wrappers for registry integration (~2-3 hours)
3. **If needed:** Enable template composition for combo creatures (~1-2 hours)

The existing code is clean, well-documented, and follows D&D 3.5e rules accurately. The architectural gap is not a bug — it's a design tradeoff that can be bridged incrementally when new features require it.